

```
In [1]: %matplotlib inline
```

神经网络实验

实验目的

掌握BP神经网络的原理和学习算法。

实验要求

编程实现N层BP神经网络算法。

实验内容1

阅读并理解以下代码。

```
In [2]: # 导入所需的模块
import numpy as np
import matplotlib.pyplot as plt

# 定义神经网络类
class NeuralNetwork:

    # 初始化网络结构和参数
    def __init__(self, input_size, hidden_size, output_size, learning_rate):
        self.input_size = input_size # 输入层节点数
        self.hidden_size = hidden_size # 隐藏层节点数
        self.output_size = output_size # 输出层节点数
        self.learning_rate = learning_rate # 学习率
        self.W1 = np.random.randn(hidden_size, input_size) # 输入层到隐藏层的权重矩阵
        self.b1 = np.random.randn(hidden_size, 1) # 隐藏层的偏置向量
        self.W2 = np.random.randn(output_size, hidden_size) # 隐藏层到输出层的权重矩阵
        self.b2 = np.random.randn(output_size, 1) # 输出层的偏置向量

    # 定义激活函数和其导数
    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self, x):
        return self.sigmoid(x) * (1 - self.sigmoid(x))

    # 定义前向传播函数, 计算网络的输出
    def forward(self, X):
        Z1 = self.W1.dot(X) + self.b1 # 隐藏层的线性组合
        A1 = self.sigmoid(Z1) # 隐藏层的激活值
        Z2 = self.W2.dot(A1) + self.b2 # 输出层的线性组合
        A2 = self.sigmoid(Z2) # 输出层的激活值, 即网络的输出
        return A2, A1, Z1

    # 定义反向传播函数, 计算网络的梯度
```

```

def backward(self, X, Y, A2, A1, Z1):
    m = X.shape[1] # 样本数
    dZ2 = A2 - Y # 输出层的误差项
    dW2 = (1 / m) * dZ2.dot(A1.T) # 输出层到隐藏层的权重梯度
    db2 = (1 / m) * np.sum(dZ2, axis=1, keepdims=True) # 输出层的偏置梯度
    dZ1 = self.W2.T.dot(dZ2) * self.sigmoid_derivative(Z1) # 隐藏层的误差项
    dW1 = (1 / m) * dZ1.dot(X.T) # 输入层到隐藏层的权重梯度
    db1 = (1 / m) * np.sum(dZ1, axis=1, keepdims=True) # 隐藏层的偏置梯度
    return dW2, db2, dW1, db1

# 定义更新函数, 根据梯度更新网络参数
def update(self, dW2, db2, dW1, db1):
    self.W2 = self.W2 - self.learning_rate * dW2 # 更新输出层到隐藏层的权重
    self.b2 = self.b2 - self.learning_rate * db2 # 更新输出层的偏置
    self.W1 = self.W1 - self.learning_rate * dW1 # 更新输入层到隐藏层的权重
    self.b1 = self.b1 - self.learning_rate * db1 # 更新隐藏层的偏置

# 定义损失函数, 计算网络的损失值
def loss(self, Y, A2):
    m = Y.shape[1] # 样本数
    L = -(Y * np.log(A2) + (1 - Y) * np.log(1 - A2)) # 交叉熵损失函数
    J = (1 / m) * np.sum(L) # 平均损失值
    return J

# 定义训练函数, 对网络进行训练和测试
def train(self, X_train, Y_train, X_test, Y_test, epochs):
    # 初始化损失值和准确率的列表, 用于记录训练过程
    train_loss = []
    train_acc = []
    test_loss = []
    test_acc = []

    # 进行指定轮数的训练
    for epoch in range(epochs):
        # 前向传播, 计算网络的输出
        A2_train, A1_train, Z1_train = self.forward(X_train)
        A2_test, A1_test, Z1_test = self.forward(X_test)

        # 反向传播, 计算网络的梯度
        dW2, db2, dW1, db1 = self.backward(X_train, Y_train, A2_train, A1_train, Z1_train)

        # 更新网络参数
        self.update(dW2, db2, dW1, db1)

        # 计算网络的损失值
        J_train = self.loss(Y_train, A2_train)
        J_test = self.loss(Y_test, A2_test)

        # 计算网络的准确率
        Y_pred_train = np.round(A2_train) # 对输出层的激活值进行四舍五入, 得到预测值
        Y_pred_test = np.round(A2_test)
        acc_train = np.mean(Y_pred_train == Y_train) # 计算预测值和真实值的一致性, 得到准确率
        acc_test = np.mean(Y_pred_test == Y_test)

        # 将损失值和准确率添加到列表中
        train_loss.append(J_train)
        train_acc.append(acc_train)
        test_loss.append(J_test)
        test_acc.append(acc_test)

```

```

# 打印每轮训练的结果
print(f"Epoch {epoch + 1}:")
print(f"Train loss: {J_train:.4f}, Train accuracy: {acc_train:.4f}")
print(f"Test loss: {J_test:.4f}, Test accuracy: {acc_test:.4f}")

# 绘制损失值和准确率的变化曲线
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.plot(train_loss, label="Train loss")
plt.plot(test_loss, label="Test loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(train_acc, label="Train accuracy")
plt.plot(test_acc, label="Test accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# 创建一个神经网络对象, 指定输入层、隐藏层、输出层的节点数和学习率
nn = NeuralNetwork(input_size=2, hidden_size=4, output_size=1, learning_rate=0.1)

# 生成一些模拟数据, 用于训练和测试
X = np.random.randn(2, 1000) # 生成1000个二维特征向量作为输入数据
Y = np.where(X[0] + X[1] > 0.5, 1, 0).reshape(1, -1) # 根据输入数据的和生成对应的标签, 1表示大于0.5, 0表示小于等于0.5

# 划分训练集和测试集, 按照8:2的比例
train_size = int(0.8 * X.shape[1]) # 训练集的大小
X_train = X[:, :train_size] # 前80%的数据作为训练集
Y_train = Y[:, :train_size]
X_test = X[:, train_size:] # 后20%的数据作为测试集
Y_test = Y[:, train_size:]

# 调用训练函数, 对网络进行训练和测试, 指定训练轮数为100
nn.train(X_train, Y_train, X_test, Y_test, epochs=100)

```

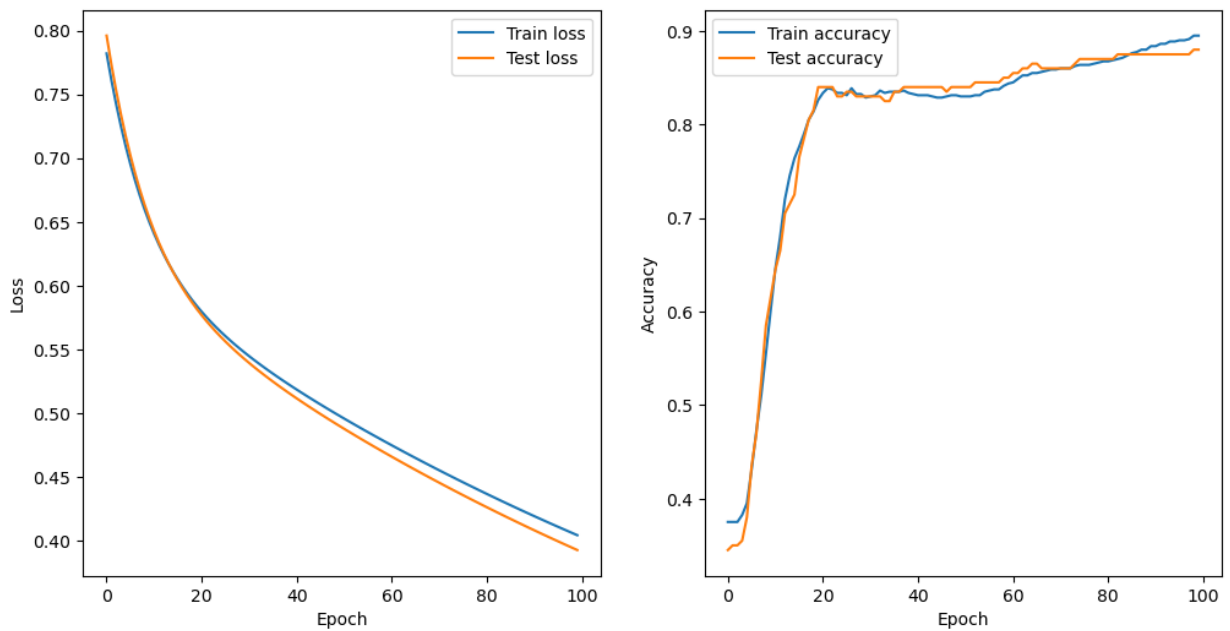
Epoch 1:
Train loss: 0.7820, Train accuracy: 0.3750
Test loss: 0.7960, Test accuracy: 0.3450
Epoch 2:
Train loss: 0.7613, Train accuracy: 0.3750
Test loss: 0.7736, Test accuracy: 0.3500
Epoch 3:
Train loss: 0.7424, Train accuracy: 0.3750
Test loss: 0.7531, Test accuracy: 0.3500
Epoch 4:
Train loss: 0.7252, Train accuracy: 0.3825
Test loss: 0.7345, Test accuracy: 0.3550
Epoch 5:
Train loss: 0.7096, Train accuracy: 0.3950
Test loss: 0.7175, Test accuracy: 0.3800
Epoch 6:
Train loss: 0.6953, Train accuracy: 0.4313
Test loss: 0.7021, Test accuracy: 0.4350
Epoch 7:
Train loss: 0.6823, Train accuracy: 0.4725
Test loss: 0.6880, Test accuracy: 0.4700
Epoch 8:
Train loss: 0.6704, Train accuracy: 0.5100
Test loss: 0.6751, Test accuracy: 0.5250
Epoch 9:
Train loss: 0.6596, Train accuracy: 0.5550
Test loss: 0.6634, Test accuracy: 0.5850
Epoch 10:
Train loss: 0.6497, Train accuracy: 0.6012
Test loss: 0.6527, Test accuracy: 0.6150
Epoch 11:
Train loss: 0.6406, Train accuracy: 0.6462
Test loss: 0.6428, Test accuracy: 0.6450
Epoch 12:
Train loss: 0.6322, Train accuracy: 0.6800
Test loss: 0.6337, Test accuracy: 0.6650
Epoch 13:
Train loss: 0.6245, Train accuracy: 0.7200
Test loss: 0.6254, Test accuracy: 0.7050
Epoch 14:
Train loss: 0.6174, Train accuracy: 0.7450
Test loss: 0.6177, Test accuracy: 0.7150
Epoch 15:
Train loss: 0.6108, Train accuracy: 0.7638
Test loss: 0.6106, Test accuracy: 0.7250
Epoch 16:
Train loss: 0.6047, Train accuracy: 0.7762
Test loss: 0.6040, Test accuracy: 0.7650
Epoch 17:
Train loss: 0.5990, Train accuracy: 0.7900
Test loss: 0.5978, Test accuracy: 0.7850
Epoch 18:
Train loss: 0.5937, Train accuracy: 0.8050
Test loss: 0.5920, Test accuracy: 0.8050
Epoch 19:
Train loss: 0.5887, Train accuracy: 0.8137
Test loss: 0.5866, Test accuracy: 0.8150
Epoch 20:
Train loss: 0.5840, Train accuracy: 0.8263
Test loss: 0.5816, Test accuracy: 0.8400

Epoch 21:
Train loss: 0.5795, Train accuracy: 0.8337
Test loss: 0.5768, Test accuracy: 0.8400
Epoch 22:
Train loss: 0.5753, Train accuracy: 0.8387
Test loss: 0.5722, Test accuracy: 0.8400
Epoch 23:
Train loss: 0.5713, Train accuracy: 0.8375
Test loss: 0.5679, Test accuracy: 0.8400
Epoch 24:
Train loss: 0.5675, Train accuracy: 0.8337
Test loss: 0.5638, Test accuracy: 0.8300
Epoch 25:
Train loss: 0.5639, Train accuracy: 0.8337
Test loss: 0.5599, Test accuracy: 0.8300
Epoch 26:
Train loss: 0.5604, Train accuracy: 0.8313
Test loss: 0.5561, Test accuracy: 0.8350
Epoch 27:
Train loss: 0.5570, Train accuracy: 0.8387
Test loss: 0.5525, Test accuracy: 0.8350
Epoch 28:
Train loss: 0.5538, Train accuracy: 0.8325
Test loss: 0.5491, Test accuracy: 0.8300
Epoch 29:
Train loss: 0.5507, Train accuracy: 0.8325
Test loss: 0.5457, Test accuracy: 0.8300
Epoch 30:
Train loss: 0.5476, Train accuracy: 0.8287
Test loss: 0.5425, Test accuracy: 0.8300
Epoch 31:
Train loss: 0.5447, Train accuracy: 0.8300
Test loss: 0.5393, Test accuracy: 0.8300
Epoch 32:
Train loss: 0.5418, Train accuracy: 0.8313
Test loss: 0.5363, Test accuracy: 0.8300
Epoch 33:
Train loss: 0.5390, Train accuracy: 0.8363
Test loss: 0.5333, Test accuracy: 0.8300
Epoch 34:
Train loss: 0.5363, Train accuracy: 0.8337
Test loss: 0.5304, Test accuracy: 0.8250
Epoch 35:
Train loss: 0.5336, Train accuracy: 0.8350
Test loss: 0.5275, Test accuracy: 0.8250
Epoch 36:
Train loss: 0.5310, Train accuracy: 0.8350
Test loss: 0.5247, Test accuracy: 0.8350
Epoch 37:
Train loss: 0.5284, Train accuracy: 0.8350
Test loss: 0.5220, Test accuracy: 0.8350
Epoch 38:
Train loss: 0.5258, Train accuracy: 0.8363
Test loss: 0.5193, Test accuracy: 0.8400
Epoch 39:
Train loss: 0.5234, Train accuracy: 0.8337
Test loss: 0.5167, Test accuracy: 0.8400
Epoch 40:
Train loss: 0.5209, Train accuracy: 0.8325
Test loss: 0.5141, Test accuracy: 0.8400

Epoch 41:
Train loss: 0.5185, Train accuracy: 0.8313
Test loss: 0.5116, Test accuracy: 0.8400
Epoch 42:
Train loss: 0.5161, Train accuracy: 0.8313
Test loss: 0.5091, Test accuracy: 0.8400
Epoch 43:
Train loss: 0.5137, Train accuracy: 0.8313
Test loss: 0.5066, Test accuracy: 0.8400
Epoch 44:
Train loss: 0.5114, Train accuracy: 0.8300
Test loss: 0.5041, Test accuracy: 0.8400
Epoch 45:
Train loss: 0.5091, Train accuracy: 0.8287
Test loss: 0.5017, Test accuracy: 0.8400
Epoch 46:
Train loss: 0.5068, Train accuracy: 0.8287
Test loss: 0.4993, Test accuracy: 0.8400
Epoch 47:
Train loss: 0.5046, Train accuracy: 0.8300
Test loss: 0.4969, Test accuracy: 0.8350
Epoch 48:
Train loss: 0.5023, Train accuracy: 0.8313
Test loss: 0.4946, Test accuracy: 0.8400
Epoch 49:
Train loss: 0.5001, Train accuracy: 0.8313
Test loss: 0.4923, Test accuracy: 0.8400
Epoch 50:
Train loss: 0.4979, Train accuracy: 0.8300
Test loss: 0.4900, Test accuracy: 0.8400
Epoch 51:
Train loss: 0.4957, Train accuracy: 0.8300
Test loss: 0.4877, Test accuracy: 0.8400
Epoch 52:
Train loss: 0.4936, Train accuracy: 0.8300
Test loss: 0.4854, Test accuracy: 0.8400
Epoch 53:
Train loss: 0.4914, Train accuracy: 0.8313
Test loss: 0.4832, Test accuracy: 0.8450
Epoch 54:
Train loss: 0.4893, Train accuracy: 0.8313
Test loss: 0.4810, Test accuracy: 0.8450
Epoch 55:
Train loss: 0.4872, Train accuracy: 0.8350
Test loss: 0.4788, Test accuracy: 0.8450
Epoch 56:
Train loss: 0.4851, Train accuracy: 0.8363
Test loss: 0.4766, Test accuracy: 0.8450
Epoch 57:
Train loss: 0.4830, Train accuracy: 0.8375
Test loss: 0.4744, Test accuracy: 0.8450
Epoch 58:
Train loss: 0.4809, Train accuracy: 0.8375
Test loss: 0.4722, Test accuracy: 0.8450
Epoch 59:
Train loss: 0.4789, Train accuracy: 0.8413
Test loss: 0.4701, Test accuracy: 0.8500
Epoch 60:
Train loss: 0.4768, Train accuracy: 0.8438
Test loss: 0.4680, Test accuracy: 0.8500

Epoch 61:
Train loss: 0.4748, Train accuracy: 0.8450
Test loss: 0.4658, Test accuracy: 0.8550
Epoch 62:
Train loss: 0.4728, Train accuracy: 0.8488
Test loss: 0.4637, Test accuracy: 0.8550
Epoch 63:
Train loss: 0.4708, Train accuracy: 0.8525
Test loss: 0.4617, Test accuracy: 0.8600
Epoch 64:
Train loss: 0.4688, Train accuracy: 0.8525
Test loss: 0.4596, Test accuracy: 0.8600
Epoch 65:
Train loss: 0.4668, Train accuracy: 0.8550
Test loss: 0.4575, Test accuracy: 0.8650
Epoch 66:
Train loss: 0.4648, Train accuracy: 0.8550
Test loss: 0.4555, Test accuracy: 0.8650
Epoch 67:
Train loss: 0.4629, Train accuracy: 0.8562
Test loss: 0.4535, Test accuracy: 0.8600
Epoch 68:
Train loss: 0.4609, Train accuracy: 0.8575
Test loss: 0.4514, Test accuracy: 0.8600
Epoch 69:
Train loss: 0.4590, Train accuracy: 0.8588
Test loss: 0.4494, Test accuracy: 0.8600
Epoch 70:
Train loss: 0.4571, Train accuracy: 0.8588
Test loss: 0.4474, Test accuracy: 0.8600
Epoch 71:
Train loss: 0.4552, Train accuracy: 0.8600
Test loss: 0.4454, Test accuracy: 0.8600
Epoch 72:
Train loss: 0.4533, Train accuracy: 0.8600
Test loss: 0.4435, Test accuracy: 0.8600
Epoch 73:
Train loss: 0.4514, Train accuracy: 0.8600
Test loss: 0.4415, Test accuracy: 0.8600
Epoch 74:
Train loss: 0.4495, Train accuracy: 0.8625
Test loss: 0.4396, Test accuracy: 0.8650
Epoch 75:
Train loss: 0.4476, Train accuracy: 0.8638
Test loss: 0.4376, Test accuracy: 0.8700
Epoch 76:
Train loss: 0.4458, Train accuracy: 0.8638
Test loss: 0.4357, Test accuracy: 0.8700
Epoch 77:
Train loss: 0.4439, Train accuracy: 0.8638
Test loss: 0.4338, Test accuracy: 0.8700
Epoch 78:
Train loss: 0.4421, Train accuracy: 0.8650
Test loss: 0.4319, Test accuracy: 0.8700
Epoch 79:
Train loss: 0.4403, Train accuracy: 0.8662
Test loss: 0.4300, Test accuracy: 0.8700
Epoch 80:
Train loss: 0.4385, Train accuracy: 0.8675
Test loss: 0.4281, Test accuracy: 0.8700

Epoch 81:
Train loss: 0.4367, Train accuracy: 0.8675
Test loss: 0.4262, Test accuracy: 0.8700
Epoch 82:
Train loss: 0.4349, Train accuracy: 0.8688
Test loss: 0.4244, Test accuracy: 0.8700
Epoch 83:
Train loss: 0.4331, Train accuracy: 0.8700
Test loss: 0.4225, Test accuracy: 0.8750
Epoch 84:
Train loss: 0.4313, Train accuracy: 0.8712
Test loss: 0.4207, Test accuracy: 0.8750
Epoch 85:
Train loss: 0.4296, Train accuracy: 0.8738
Test loss: 0.4189, Test accuracy: 0.8750
Epoch 86:
Train loss: 0.4278, Train accuracy: 0.8762
Test loss: 0.4170, Test accuracy: 0.8750
Epoch 87:
Train loss: 0.4261, Train accuracy: 0.8775
Test loss: 0.4152, Test accuracy: 0.8750
Epoch 88:
Train loss: 0.4243, Train accuracy: 0.8800
Test loss: 0.4134, Test accuracy: 0.8750
Epoch 89:
Train loss: 0.4226, Train accuracy: 0.8800
Test loss: 0.4117, Test accuracy: 0.8750
Epoch 90:
Train loss: 0.4209, Train accuracy: 0.8838
Test loss: 0.4099, Test accuracy: 0.8750
Epoch 91:
Train loss: 0.4192, Train accuracy: 0.8838
Test loss: 0.4081, Test accuracy: 0.8750
Epoch 92:
Train loss: 0.4175, Train accuracy: 0.8862
Test loss: 0.4064, Test accuracy: 0.8750
Epoch 93:
Train loss: 0.4158, Train accuracy: 0.8862
Test loss: 0.4046, Test accuracy: 0.8750
Epoch 94:
Train loss: 0.4142, Train accuracy: 0.8888
Test loss: 0.4029, Test accuracy: 0.8750
Epoch 95:
Train loss: 0.4125, Train accuracy: 0.8888
Test loss: 0.4012, Test accuracy: 0.8750
Epoch 96:
Train loss: 0.4109, Train accuracy: 0.8900
Test loss: 0.3995, Test accuracy: 0.8750
Epoch 97:
Train loss: 0.4092, Train accuracy: 0.8900
Test loss: 0.3978, Test accuracy: 0.8750
Epoch 98:
Train loss: 0.4076, Train accuracy: 0.8912
Test loss: 0.3961, Test accuracy: 0.8750
Epoch 99:
Train loss: 0.4060, Train accuracy: 0.8950
Test loss: 0.3944, Test accuracy: 0.8800
Epoch 100:
Train loss: 0.4044, Train accuracy: 0.8950
Test loss: 0.3927, Test accuracy: 0.8800



上机内容2

1. 阅读相关资料，了解BP神经网络的原理和算法。
2. 设计一个3层BP神经网络，确定输入层、隐藏层和输出层的节点数，以及激活函数和误差函数的选择。
3. 编写代码，实现网络的初始化、前向传播、反向传播和权值更新等功能。**要求模块化设计代码，可以通过参数调节网络层数、宽度等。**
4. 选择iris数据集，对网络进行训练和测试，记录网络的误差变化和准确率变化，**提高网络在测试集的准确率。**
5. 修改网络的参数，重复上一步步骤，比较不同参数下的网络性能。
6. 撰写实验报告，总结实验结果和心得体会。

In [3]: `%reset -f`